CustomerID	transactionCount	eventTimestamp	isTimeout
5	3	2023-09-03T19:24:14.388	false
2	4	2023-09-03T19:24:16.721	true
3	0	2023-09-03T19:24:16.720	true

Figure 5.8 – This table is the result after applying the stateful streaming transformation

The feature table shown in *Figure 5.8* includes the `CustomerID` reference, `transactionCount` for that customer, `eventTimestamp`, and a Boolean variable, `isTimeout`. The `eventTimestamp` is the time the feature record was written. We call it `eventTimestamp` because a new transaction or a timeout could have triggered the update to the customer state/event. To know which type of event it is, we include `isTimeout`. A timeout occurs when no new transactions for a customer have occurred but the value of `transactionCount` has changed – an indication that the count has decreased.

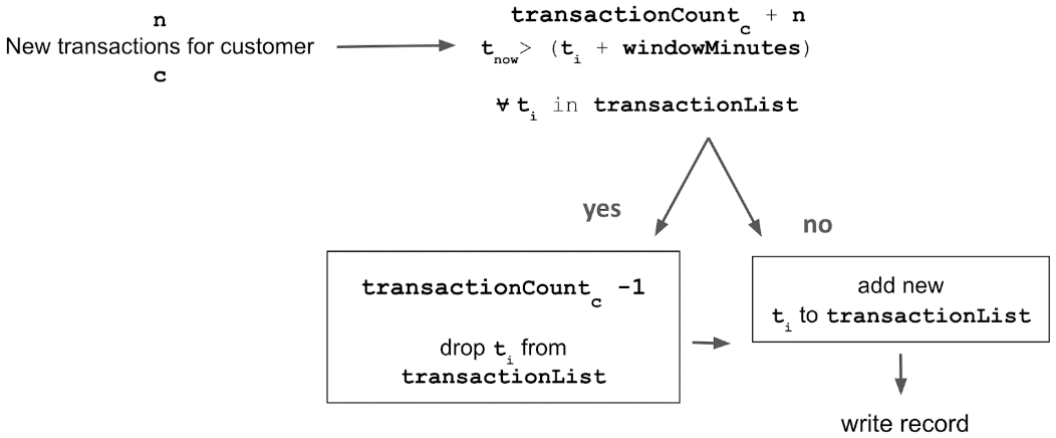


Figure 5.9 – Logic flow for the stateful streaming transformation

Figure 5.9 visually represents the update logic applied to the customer state. The logical path can be broken into steps:

1. For `n` new transactions coming in for customer `c`, `transactionCount` is incremented `n` times.
2. Then, for each `transactionTimestamp`, `ti`, in `transactionList`, we compare the current time with `expirationTimestamp` (`ti+windowMinutes`) to determine whether the transaction accounted for in `transactionCount` has expired:

    I. If any transactions have expired, we decrement `transactionCount` by one for each and drop the corresponding `transactionTimestamp` from `transactionList`.